

Deep learning – Tools and Tricks

I wish I had known earlier

Peter Lippmann

05.02.2026

Scientific AI Lab, IWR, Heidelberg University

Deep learning overview

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} L(y, f_{\theta}(x))$$

$\mathcal{D}$: data distribution, typically approximated by a finite set of input-target pairs $\{x_i, y_i\}$

f_{θ} : neural network with learnable parameters θ

L : differentiable loss function, typically minimal if $y = f_{\theta}(x)$

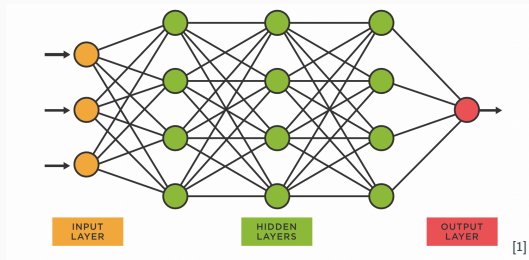
In practice:

$$\min_{\theta} \sum_{\substack{\text{sample } (x_i, y_i) \\ \text{in training set}}} L(y_i, f_{\theta}(x_i))$$

Optimization via (mini batch) gradient descent:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \sum_{\substack{\text{sample } (x_i, y_i) \\ \text{in mini batch}}} \nabla_{\theta} L(y_i, f_{\theta}(x_i))$$

The neural network



- Network types: CNNs for images, GNNs for graphs, sequence models for language, recurrent NNs for time series data, ... (often clear what to choose)
- BUT many representatives, e.g. many different CNN architectures (less clear)
- For given architecture, several hyperparameters (educated guess + trying out)

The loss function

$$\min_{\theta} \sum_{\substack{\text{sample } (x_i, y_i) \\ \text{in training set}}} L(y_i, f_{\theta}(x_i))$$

- Heavily depends on your task, e.g.:
 - classification $\leftrightarrow$ Cross Entropy loss,
 - regression $\leftrightarrow$ MSE (a.k.a. L2-loss) $\|y_i - f_{\theta}(x_i)\|^2$
- Tip: for regression L1-loss $|y_i - f_{\theta}(x_i)|$ can be more robust to outliers
- Sometimes a combination of losses is used (weighting them can be tricky)

The data

- More data is better, higher quality data is better
- Visualize your data before: PCA, UMAP, t-SNE
- Check your data is balanced (e.g. in instances per class)
- Split your data into train, validation and test set
- Standardize your data (both input and output)
- Use data augmentation if possible



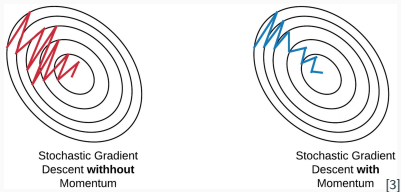
[2]

Data augmentation

The optimizer

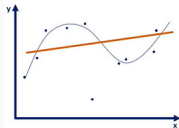
Use AdamW (Adaptive Moment Estimation + weight decay):

- Adaptive learning rate helps against too small or too large gradients
- Momentum stabilizes the gradient descent
- Weight decay helps against overfitting

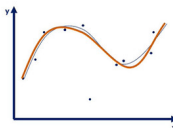


(a) Effect of momentum

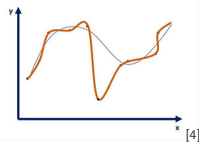
An **underfitted** model



A **good** model

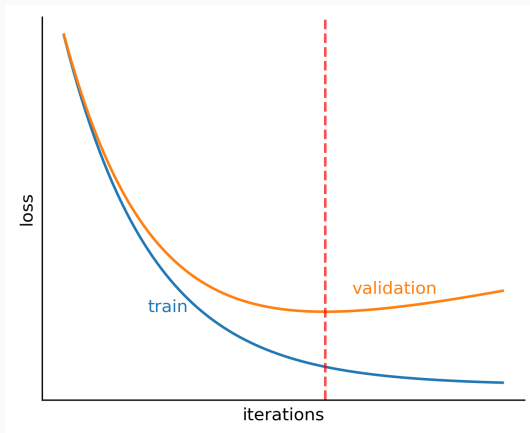


An **overfitted** model



(b) Overfitting model

More tricks against overfitting



- Dropout: randomly drop/ignore neurons during training
- Save checkpoints of your model: last (if training crashes) & k -many best

More Training Tricks

- Try to overfit on a single sample to debug your pipeline
- Set a seed during training for reproducibility
- Use enough CPU workers in dataloader to properly use GPU
- In dataloader use `shuffle=True` and `drop_last=True`
- Try gradient clipping in the optimizer against instable training
- Use a learning rate scheduler (e.g. cosine schedule)
- Try learning rate warmup
- Definitely try normalization layers: helps to standardize activations

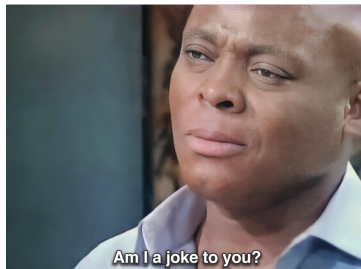
Model evaluation

- During training, evaluate your model on validation set
- After training (when ready to publish), evaluate your model ONCE on the TEST set
- Use suitable metric to benchmark your model (e.g. accuracy for classification, see <https://metrics-reloaded.dkfz.de/> for image analysis)

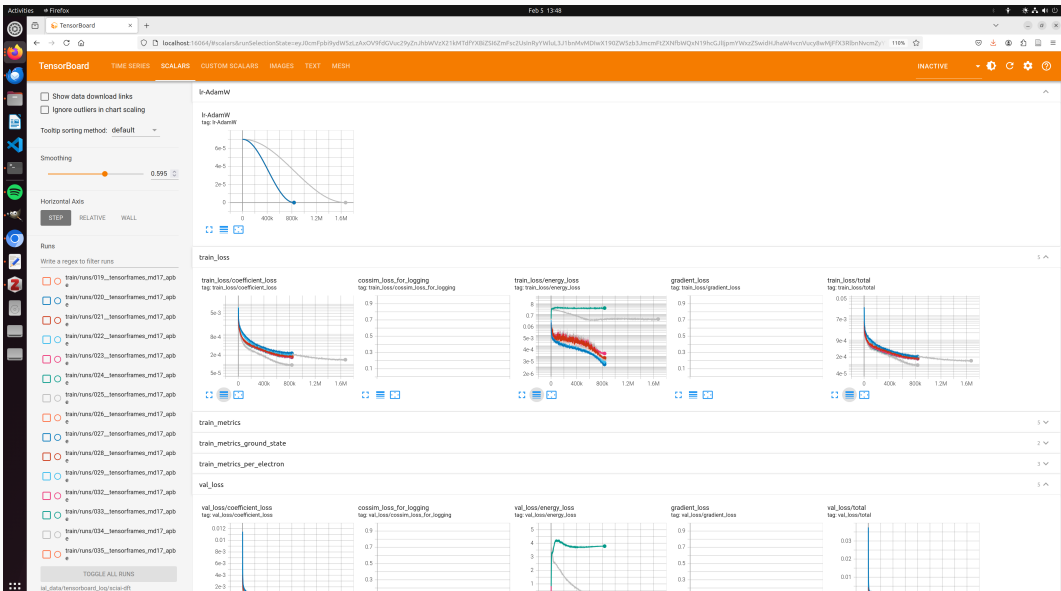
Official implementation of the NeurIPS 23 spotlight paper of  InfGCN .

```
Code Blame
20 @register_dataset('small_density')
21 class SmallDensityDataset(Dataset):
22     def __init__(self, root, mol_name, split):
23         """
24         Density dataset for small molecules in the MD datasets.
25         Note that the validation and test splits are the same.
26         :param root: data root
27         :param mol_name: name of the molecule
28         :param split: data split, can be 'train', 'validation', 'test'
29         """
30         super(SmallDensityDataset, self).__init__()
31         assert mol_name in ('benzene', 'ethanol', 'phenol', 'resorcinol', 'ethane', 'n
32         self.root = root
33         self.mol_name = mol_name
34         self.split = split
35         if split == 'validation':
36             split = 'test'
37
38         self.n_grid = 50 # number of grid points along each dimension
39         self.grid_size = 20. # box size in Bohr
40         self.data_path = os.path.join(root, mol_name, f'{mol_name}_{split}')
```

Scientific Integrity



Logging via TensorBoard



Pytorch Lightning

Pytorch lightning module combines pytorch model + optimizer + logging

- Abstracts away to ("cuda"), loss.backward(), model.eval() and much more
- Makes complicated things which many people use easy, e.g. multi GPU support

```
import lightning as L
import torch

from lightning.pytorch.demos import Transformer

class LightningTransformer(L.LightningModule):
    def __init__(self, vocab_size):
        super().__init__()
        self.model = Transformer(vocab_size=vocab_size)

    def forward(self, inputs, target):
        return self.model(inputs, target)

    def training_step(self, batch, batch_idx):
        inputs, target = batch
        output = self(inputs, target)
        loss = torch.nn.functional.nll_loss(output, target.view(-1))
        return loss

    def configure_optimizers(self):
        return torch.optim.SGD(self.model.parameters(), lr=0.1)
```

Many great tutorials at

<https://lightning.ai/docs/pytorch/stable/starter/introduction.html>

Config management with Hydra

- Save on boilerplate by “programming” in configs (customize models in config not in code)

```
net:
  _target_: mldft.ml.models.components.graphformer.Graphformer
  edge_mlp:
    _target_: mldft.ml.models.components.mlp.MLP
    in_channels: 128
    hidden_channels: [768, 32]
    activation_layer:
      _target_: hydra.utils.get_class
      path: torch.nn.SiLU
    dropout: 0.
  energy_mlp:
    _target_: mldft.ml.models.components.mlp.MLP
    in_channels: 768
    hidden_channels: [768, 1]
    activation_layer:
      _target_: hydra.utils.get_class
      path: torch.nn.SiLU
    dropout: 0.
    disable_dropout_last_layer: True
    disable_activation_last_layer: True
    disable_norm_last_layer: True
```

- Easy to add new models, datasets, tasks and experiments
- Uses OmegaConf for configuration management

Great Hydra + Lightning template at

<https://github.com/ashleve/lightning-hydra-template>

How I use LLMs in my workflow

- Github co-pilot auto-completion in VS code
- Agent mode or LLMs for debugging: just paste the error traceback
- LLMs for speeding up code: from for-loops to vectorized code

```
# kNN implementation with for-loop:
import torch

k = 3
x = torch.rand(10000, 5) # 100 samples, 5 features
knn_indices = []
for x_i in x:
    # Compute distances from x_i to all other points
    distances = torch.norm(x - x_i, dim=1)

    # sort distances and get indices of k nearest neighbors
    knn_idx = torch.argsort(distances)[:k+1] # +1 to exclude self
    knn_indices.append(knn_idx[1:]) # Exclude self

knn_indices = torch.stack(knn_indices)
✓ 9.8s
```



```
# Pairwise squared Euclidean distances via broadcasting: (N, N)
# dist2[i, j] = ||x[i] - x[j]||^2
diff = x[:, None, :] - x[None, :, :] # (N, N, D)
dist2 = (diff * diff).sum(dim=2) # (N, N)

# Exclude self by setting diagonal to +inf
dist2.fill_diagonal_(float("inf"))

# Full sort per row, take k nearest
knn_indices_parallel = dist2.argsort(dim=1)[: , :k] # (N, k)
✓ 0.6s
```

And check:

```
torch.allclose(knn_indices, knn_indices_parallel) # Should be True
✓ 0.0s
True
```

Caveat: Bugs introduced by LLMs are hard to spot! ALWAYS check LLM code line by line!

Thank You!

Any Questions?

Image References

- [1] <https://www.marktechpost.com/wp-content/uploads/2022/09/Screen-Shot-2022-09-23-at-10.46.58-PM.png>
- [2] https://media.datacamp.com/legacy/image/upload/v1669203370/Data_Augmentation_Header_f42227f2cb.png
- [3] <https://i.sstatic.net/epW89.jpg>
- [4] https://static.wixstatic.com/media/0ed3e8_a9b7d6d3dc6b4d5cbcb30c8b2fd4782b~mv2.jpg/v1/fill/w_1000,h_449,al_c,q_90,usm_0.66_1.00_0.01/0ed3e8_a9b7d6d3dc6b4d5cbcb30c8b2fd4782b~mv2.jpg